

Transformers

The Transformer architecture [Vaswani et al, NeurIPS 2017] has improved the performance of neural language models in a very significant manner compared to previous neural architectures (RNNs, LSTMs, GRUs) which were developed as a replacement of n-gram models.

In this approach, the probability p is modeled using a transformer architecture with parameters θ . The transformer architecture includes the following components, adopting the notation used in [Radford et al, 2018]:

- Initial token embedding: $h_0 = UW_e + W_p$
- Multi-layer transformation: $h_l = \text{transformer_block}(h_{l-1}) \forall l \in [1, n]$
- Final prediction for the next token: $p(u) = \text{softmax}(h_n W_e^T)$

where $U = (u_{-k}, \dots, u_{-1})$ is the context vector of the last k tokens, n is the number of layers, W_e is the token embedding matrix, and W_p is the position embedding matrix. The initial embeddings can be adopted from a context-free embedding method such as Word2Vec or Glove. The softmax function turns a vector of size V which scores each candidate word (V is number of distinct tokens in the tokenizer vocabulary) into a distribution (a vector of V dimensions with all values between 0 and 1 and the sum of the values is 1).

Positional Embeddings (W_p)

Positional embeddings are used in transformer architectures to provide information about the position of a word in the input sequence. Without positional embeddings, the model would not be able to distinguish the order of the words in the sequence, as the self-attention mechanism used in transformers does not take the order of the input into account. Positional embeddings help the model understand the relationship between words in the context of their position in the input, allowing it to make more accurate predictions.

Several forms of positional embeddings are commonly used in LLMs, including:

- **Sinusoidal Positional Encoding:** This method represents each position in the input sequence as a combination of sine and cosine functions with varying frequencies.
- **Learned Positional Embeddings:** In this approach, the model learns the positional embeddings during training, rather than using pre-defined encodings.

The most commonly used form of Positional Embedding is currently the [RoPE method](#) (2021).

Transformer Block: Self-Attention

A transformer block for a single layer in the transformer architecture consists of a multi-head **self-attention mechanism** and a feed-forward neural network, connected by layer normalization and residual connections.

Attention describes the mutual influence of tokens within the sequence. The representation of each token is a weighted combination of the representation of the neighboring tokens. Learning an attention mechanism means we learn how to compute the influence weights given a specific sequence of tokens.

The equations for the transformer block can be broken down into several steps:

- **Input Embedding:** The input to the transformer block is first passed through an embedding layer, which projects the input into a higher-dimensional space. This can be represented by the equation: $X = \text{Embedding}(X)$
- **Multi-Head Self-Attention:** The embedded input is then passed through the multi-head self-attention mechanism, which calculates the attention weights between different positions in the input sequence. This is done by first calculating the dot product of the embedded input with three weight matrices (W_Q, W_K, W_V) to get the query, key, and value matrices, respectively. Then, the dot product of the query and key matrices is scaled and passed through a softmax function to obtain the attention weights. The attention weights are then used to calculate the weighted sum of the value matrix, which is the output of the multi-head self-attention mechanism. This process can be represented by the following equations:

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V$$

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \text{sqrt}(d_k))V$$

- **Layer Normalization:** The output of the multi-head self-attention mechanism is then passed through a layer normalization step, which normalizes the values to have zero mean and unit variance. This can be represented by the equation:

$$X_{\text{norm}} = \text{LayerNorm}(\text{Attention}(Q, K, V))$$

- **Feed-Forward Neural Network:** The output of the layer normalization step is then passed through a feed-forward neural network, which consists of two linear layers with a ReLU activation function in between. This can be represented by the following equations:

$$FF = \text{ReLU}(X_{\text{norm}}W_1 + b_1)W_2 + b_2$$

- **Residual Connection:** Finally, the output of the feed-forward neural network is added to the input of the transformer block with a residual connection, which helps to preserve the information from the input. This can be represented by the equation:

$$X_{\text{out}} = X_{\text{norm}} + FF$$

- **Final Output:** The final output is passed through a final linear layer and a softmax function to obtain the probability distribution over the vocabulary for each position in the

input sequence.

Transformer References

The following two blog posts present complementary perspectives on the paper which introduced the Transformer architecture [Vaswani et al, NeurIPS 2017]:

Attention Is All You Need
Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin, NeurIPS 2017
https://dl.acm.org/doi/10.5555/3295222.3295349

- [The Illustrated Transformer](#) (Jay Alammar 2020)

This post provides a self-contained presentation of the technology which underlies most Pretrained Language Models (LLMs): transformers. It provides an intuitive understanding of how Transformer neural models are trained and evaluated.

- [The Annotated Transformer](#) (Sasha Rush 2020)

This page originally written by Sasha Rush is a code annotated version of the Attention paper. It explains the article by developing PyTorch code which implements the Transformer architecture, and illustrates how to use the code in a learning and inference mode. It is a hands-on, efficient manner to learn and understand this essential technique.